Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 1

з дисципліни «Системне програмування» на тему:

«Налаштування середовища розробки та bring-up платформи STM32»

Виконали:
студенти групи ІО-32
Крадожон М. Р.
Пилипчук В. Д.
студенти групи ІО-35
Комар В. О.
Сабадаш В. Г.
Хімко А. Р.

Перевірив:
Гордієнко Н. Ю.

Київ — 2026 р.

Мета роботи:

Метою лабораторної роботи було підготувати команду та апаратно-програмне середовище до подальшої розробки вбудованої системи на базі STM32. На початковому етапі ми розглядали лабораторну насамперед як налаштування відтворюваного середовища розробки, побудову проєкту, його прошивку на реальну плату та отримання першого результату роботи через UART. У процесі виконання мета розширилася до практичного ознайомлення зі структурою STM32-проєкту, конфігурацією периферійних інтерфейсів і підготовкою апаратної платформи для реалізації обраного командного проєкту.

Окремим результатом роботи стало формування основи для командної розробки в наступних лабораторних роботах. Ми відпрацювали спільну роботу з репозиторієм, розподіл відповідальності між учасниками, взаємодію під час налаштування середовища та узгодження технічних рішень. Таким чином, наприкінці лабораторної ми мали не лише працездатну STM32-платформу, а й узгоджене розуміння її можливостей, способів роботи з підключеними компонентами та організації подальшої розробки мультирежимного реєстратора інцидентів.

Розподіл обов'язків у команді:

Таблиця 1

Учасник/ця	Роль	Внесок	Артефакти	Коміти / PR
Артем Хімко	Hardware Research Engineer	1. Опрацьовано технічну документацію та підібрано datasheet-и для MPU-6050, W25Q64JV та OV7670. 2. Досліджено основні характеристики компонентів, інтерфейси, адреси, режими роботи, живлення та необхідні регістри для первинної перевірки. 3. Підготовлено docs/hardware-components.md з таблицями підключення компонентів до NUCLEO-F411RE.	docs/hardware-components.md	PR #4

Продовження таблиці 1

Учасник/ця	Роль	Внесок	Артефакти	Коміти / PR
Вікторія Сабадаш	CI/CD Engineer	1. Налаштовано автоматичний CI-конвеєр у GitHub Actions для перевірки збірки проєкту. 2. Реалізовано запуск workflow на кожен push та pull_request у гілку main, з виключенням змін, що стосуються лише документації. 3. Додано автоматичне створення Docker-образу та запуск make build для перевірки компіляції прошивки. 4. Додано перевірку наявності результатів збірки (.bin, .elf, .hex) та збереження їх як CI-артефактів.	.github/ workflows/ ci.yml	PR #3
Максим Крадожон	Team Lead / Firmware & Bring-Up	1. Налаштовано базовий проєкт, дерево тактування (84 МГц) та периферію (USART2, I2C1, SPI2, GPIO LD2) в STM32CubeMX. 2. Реалізовано перевизначення системного виклику _write() для роботи printf() через USART2. 3. Налаштовано локальне середовище збірки мікропрограми на базі Docker-контейнера з arm-none-eabi-gcc. 4. Виконано прошивку плати NUCLEO-F411RE за допомогою утиліти st-flash. 5. Зібрано апаратний стенд: підключено MPU-6050 (I2C1 на PB8/PB9, пін адреси AD0 до GND) та Flash-пам'ять W25Qxx (SPI2 на PB12–PB15) через макетну плату з живленням 3.3 В. 6. Перевірено стабільність UART-зв'язку (115200 бод) та системного таймінгу HAL_GetTick().	firmware/ firmware.ioc firmware/ Makefile firmware/Core/ Src/main.c firmware/ STM32F411xx_ FLASH.ld	c94ebb4 7961f7f 8257dc7 d366dfa

Кінець таблиці 1

Учасник/ця	Роль	Внесок	Артефакти	Коміти / PR
Владислав Комар	Product & Documentation Lead	1. Запропоновано команді тему проєкту — «Мультирежимний реєстратор інцидентів (Incident Recorder: Cargo + Site)»; узгоджено концепцію та основні сценарії використання пристрою. 2. Розроблено PRD із 17 функціональними вимогами FR-01–FR-17, обґрунтуванням вибору компонентів, розглядом альтернатив та планом тестування T-01–T-09. 3. Підготовлено фінальний звіт: сформовано структуру, зібрано матеріали від учасників команди, оформлено титульний аркуш, описано процес виконання та проблеми, підготовлено фінальну версію у форматі PDF.	docs/PRD.md docs/reports/ report_1.pdf	PR #2 PR #5
Вікторія Пилипчук	DevOps & Build System	1. Створено ізольоване Docker-оточення на базі Ubuntu 22.04 з необхідним ARM GNU Toolchain та допоміжними інструментами. 2. Реалізовано кореневий Makefile з автоматизованими командами make build, make clean, make flash та make monitor. 3. Налаштовано збірку прошивки всередині Docker-контейнера із збереженням правильного власника створених артефактів. 4. Створено .gitignore для виключення бінарних, об’єктних і тимчасових файлів збірки та файлів IDE.	docker/ Dockerfile Makefile .gitignore	e35aa6c

Виконання лабораторної роботи:

1. Формування концепції проєкту та початкове планування

На початковому етапі ми визначили концепцію майбутнього командного проєкту — «Мультирежимний реєстратор інцидентів (Incident Recorder: Cargo + Site)». Основна ідея полягала у створенні єдиної апаратно-програмної платформи, яка може працювати у двох режимах: Cargo для контролю умов транспортування вантажу та Site для контролю стаціонарного об’єкта. Нами було визначено основні сценарії використання системи, перелік контрольованих параметрів та загальну роль кожного компонента в майбутньому продукті.

Після узгодження концепції ми розподілили між учасниками команди основні напрямки роботи: DevOps & Build System, CI/CD Engineer, Team Lead / Firmware & Bring-Up, Hardware Research Engineer та Product & Documentation Lead. Такий розподіл дозволив паралельно опрацьовувати програмне середовище, апаратну частину та документацію. На цьому ж етапі було сформовано базову структуру репозиторію та визначено, які результати кожного напрямку мають бути представлені у вигляді окремих артефактів.

Паралельно Product & Documentation Lead працював над документом docs/PRD.md (PR #2), у якому було зафіксовано концепцію продукту, функціональні вимоги FR-01–FR-17, перелік компонентів, їх альтернативи та план тестування T-01–T-09.

2. Підготовка репозиторію та ізольованого середовища збірки

Наступним етапом ми налаштували єдине середовище розробки, у якому прошивка могла збиратися незалежно від локальної конфігурації комп’ютера. DevOps & Build System створила docker/Dockerfile на базі Ubuntu 22.04 та додала до нього ARM GNU Toolchain і необхідні допоміжні засоби. Після цього було реалізовано кореневий Makefile, який забезпечив стандартний набір команд для роботи з проєктом:

— make build — збірка прошивки всередині Docker-контейнера;
— make clean — очищення результатів збірки;
— make flash — прошивка плати через st-flash;
— make monitor — запуск UART-монітора через picocom.

Також було створено .gitignore, до якого додано правила для виключення об'єктних файлів, бінарних артефактів, результатів збірки та службових файлів IDE. Основними артефактами цього етапу стали docker/Dockerfile, Makefile та .gitignore (коміт e35aa6c).

3. Автоматизація перевірки збірки через CI/CD

Після перевірки локальної збірки ми автоматизували її контроль за допомогою GitHub Actions. CI/CD Engineer створив workflow .github/workflows/ci.yml, який запускається при виконанні push, а також для pull_request у гілку main. Таким чином, кожна зміна програмного коду автоматично перевіряється ще до її інтеграції в основну гілку.

У межах workflow виконується послідовність дій:

— завантаження актуального стану репозиторію;
— створення Docker-образу stm32-build;
— запуск make build;
— перевірка наявності .bin, .elf та .hex;
— збереження результатів збірки як CI-артефактів.

Щоб не запускати збірку без потреби, для змін, що стосуються лише документації (docs/**, *.md, *.pdf), було додано paths-ignore.

Налаштований CI-конвеєр додано до репозиторію через PR #3 (.github/workflows/ci.yml). Його використання дозволить перевіряти працездатність build-системи автоматично та зменшити ризик потрапляння непрацюючих змін до основної гілки.

4. Конфігурація STM32 у STM32CubeMX

Після підготовки середовища збірки ми перейшли до конфігурації апаратної платформи NUCLEO-F411RE у STM32CubeMX. Team Lead / Firmware & Bring-Up створив базовий проєкт та налаштував необхідні периферійні модулі відповідно до запланованої архітектури системи. Основним завданням на цьому етапі було отримати правильно налаштований HAL-скелет прошивки з необхідними інтерфейсами для подальшого підключення сенсорів і зовнішньої пам’яті.

У конфігурації було задіяно I2C1 на PB8/PB9, SPI2 на PB12–PB15, а також USART2 на PA2/PA3. Додатково було налаштовано GPIO PA5 для керування вбудованим світлодіодом LD2, який надалі використовується для базової індикації стану системи. Відповідну конфігурацію виводів наведено на рис. 1.

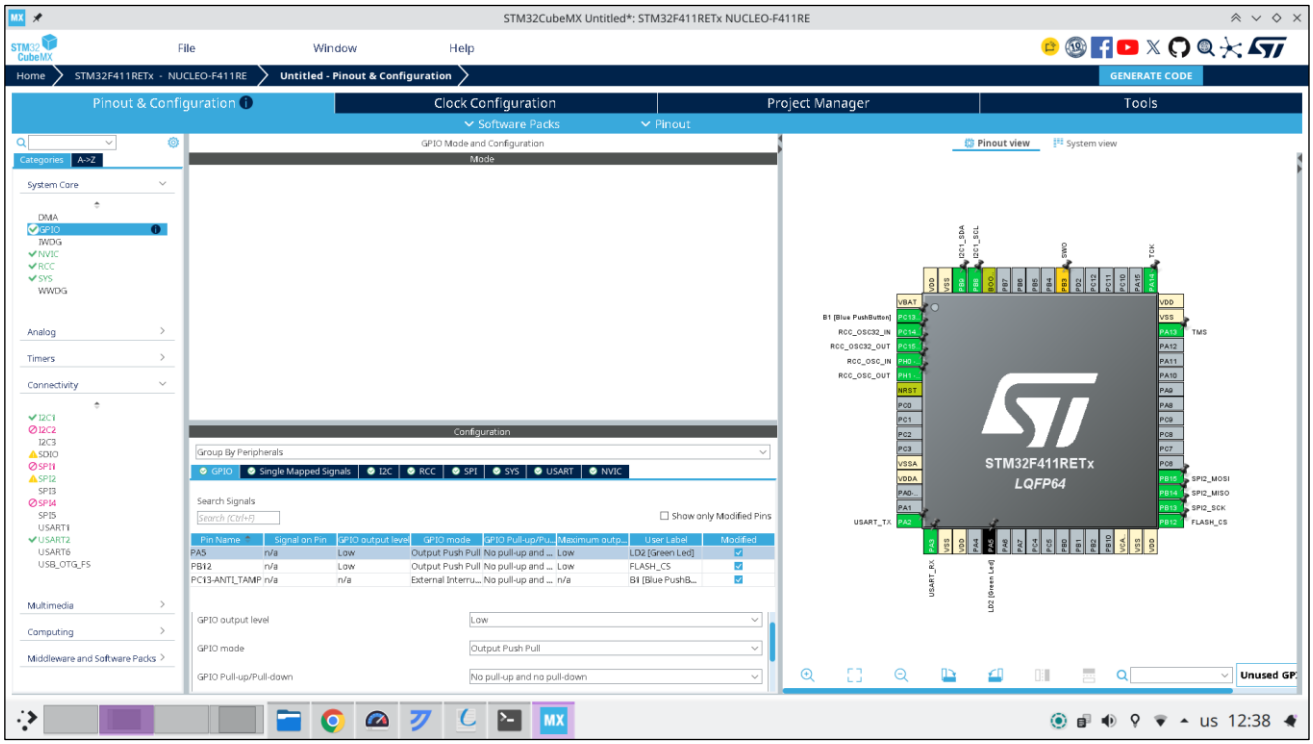


Рисунок 1

Під час налаштування тактування було сформовано дерево системних частот із SYSCLK = 100 МГц, після чого STM32CubeMX автоматично розрахував пов’язані частоти шин та периферійних модулів. Це дозволило отримати узгоджену конфігурацію тактування перед генерацією програмного коду. Налаштоване дерево

тактування показано на рис. 2. Після завершення конфігурації було згенеровано проєкт із HAL-ініціалізацією та збережено відповідний файл firmware.ioc.

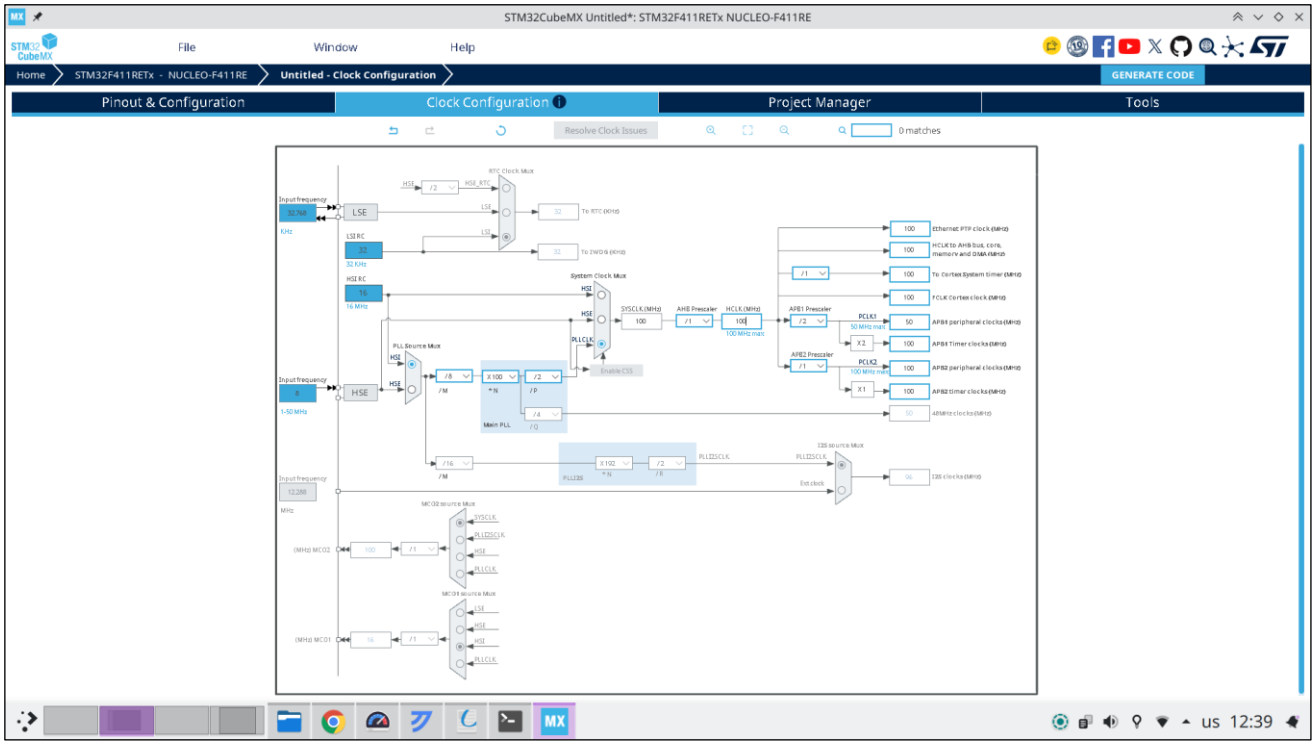


Рисунок 2

Результатом цього етапу став базовий firmware-проєкт із підготовленими модулями USART2, I2C1, SPI2 та GPIO. Зміни конфігурації STM32CubeMX і пов'язані з ними файли були внесені Team Lead / Firmware & Bring-Up до репозиторію (коміт d366dfa).

5. Реалізація UART-виводу та першого запуску прошивки

Після генерації початкового коду ми реалізували базовий механізм виводу діагностичних повідомлень через USART2. Для цього Team Lead / Firmware & Bring-Up реалізував перевизначення системного виклику _write(), через яке стандартний printf() передає дані до UART. У програмі також було використано системний таймер HAL для періодичного виводу поточного значення HAL_GetTick(), що дозволило одночасно перевірити працездатність UART-зв'язку та системного таймінгу.

Виконавши компіляцію, отриману прошивку було записано до NUCLEO-F411RE за допомогою st-flash, після чого був відкритий UART-монітор через picocom зі швидкістю 115200 бод. У терміналі отримано стабільний послідовний вивід повідомлень із оновленням системного лічильника, що підтвердило правильну ініціалізацію HAL, тактування, USART2 та виконання основного циклу програми. Результат першого запуску наведено на рис. 3.

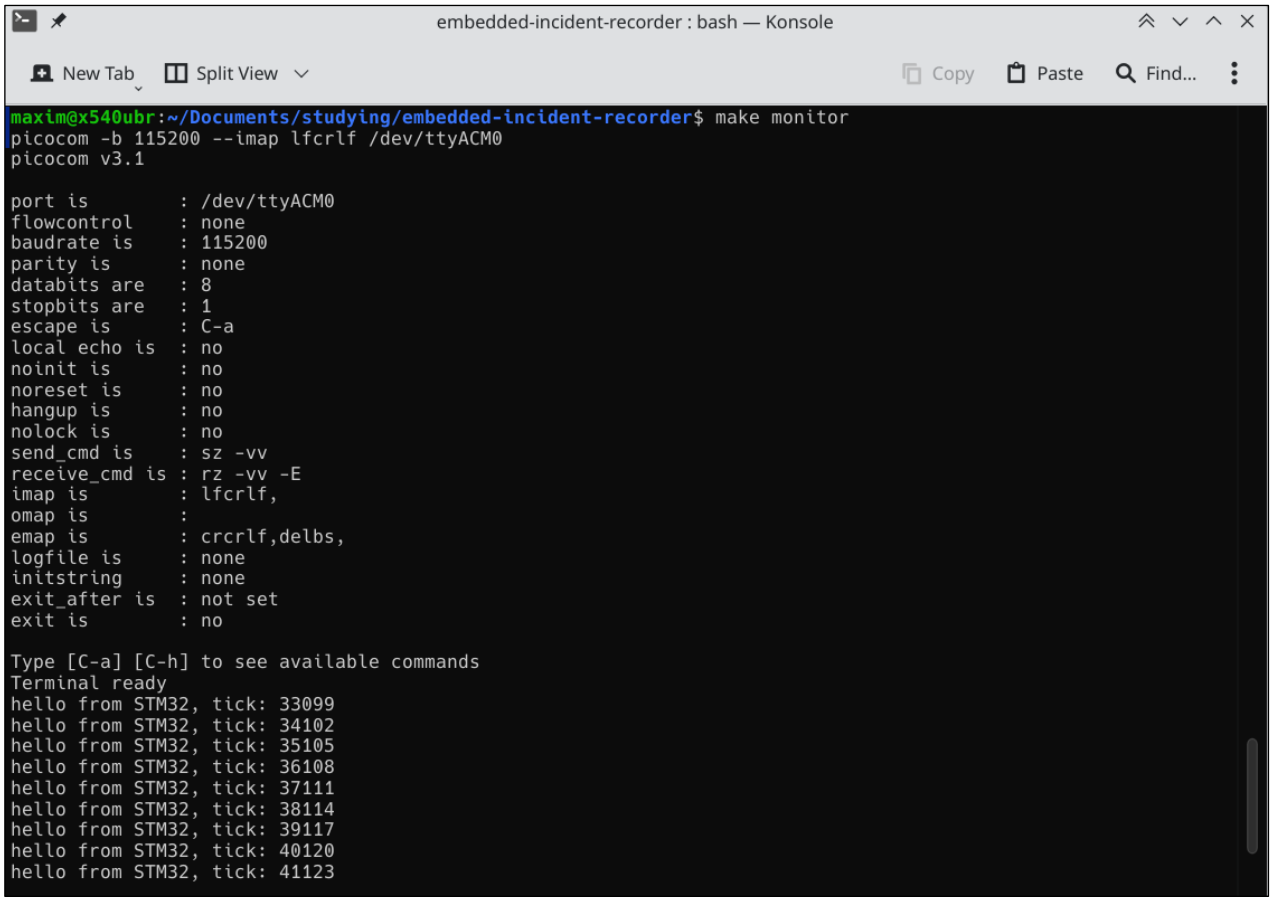


Рисунок 3

6. Монтаж апаратного стенду та підключення периферії

Загальний вигляд зібраного стенду наведено на рис. 4, а відео з демонстрацією розміщене за посиланням. На ньому показано плату NUCLEO-F411RE, підключений модуль MPU-6050 та зовнішню Flash-пам’ять на макетній платі. Team Lead / Firmware & Bring-Up виконав фізичне підключення компонентів і первинну перевірку стенду, а Hardware Research Engineer забезпечив відповідність

підключення технічним параметрам компонентів, зафіксованим у docs/hardware-components.md (PR #4). Відео з демонстрацією роботи

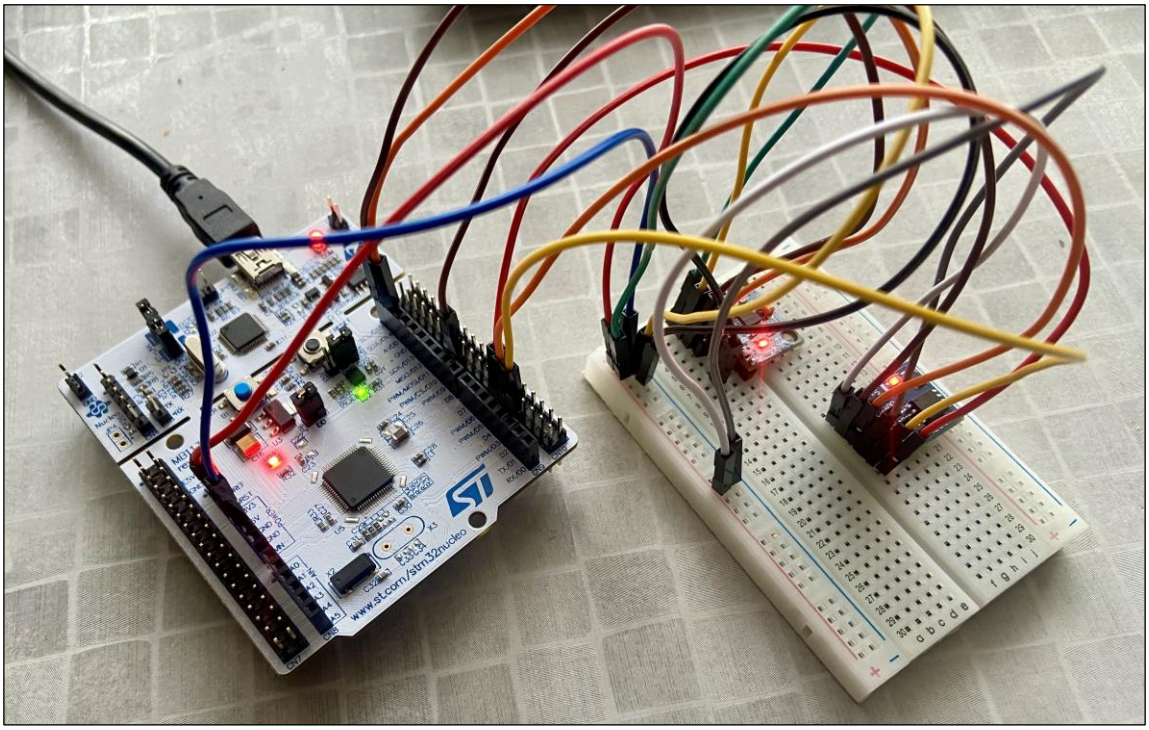


Рисунок 4

7. Аналіз проблем та їх вирішення

У ході виконання лабораторної роботи виникали проблеми технічного характеру, які описані нижче.

1) Блокування збереження конфігурації в STM32CubeMX через права доступу в Linux

Суть проблеми: Під час генерації коду або спроби зберегти файл конфігурації (.ioc) утиліта STM32CubeMX видавала помилку запису, сигналізуючи про режим «read-only» для каталогу проєкту. Це сталося через конфлікт прав доступу користувача на локальній файловій системі Linux.

Вирішення: Було перевірено та скориговано права доступу на кореневу папку проєкту й підкаталоги (командами зміни прав володіння chown / дозволів chmod), після чого конфігурація периферії збереглася та згенерувала структуру вихідного коду без збоїв.

2) Відсутність готового Docker-образу з тулчейном у віддаленому реєстрі

Суть проблеми: При спробі виконати збірку проєкту згідно з Makefile система намагалася затягнути попередньо скомпільований образ крос-компілятора (arm-none-eabi-gcc), якого не виявилося в публічному Docker-реєстрі (помилка manifest not found / відсутній тег).

Вирішення: Замість завантаження зовнішнього образу було виконано локальну збірку Docker-образу з наданого Dockerfile (docker build -t ...), після чого процес компіляції прошивки в контейнері відпрацював штатно, сформувавши цільовий бінарний файл .bin.

3) Помилка лінковки та конфлікт декларацій функції printf()

Суть проблеми: Після підключення стандартного заголовного файлу <stdio.h> для виклику printf() компілятор видавав помилки/попередження щодо конфлікту типів і сигнатур при спробі перевизначити функцію виводу, а рядок не передавався в апаратний буфер UART.

Вирішення: Було належним чином реалізовано низькорівневий POSIX-хук int _write(int file, char *ptr, int len) із передачею символів через HAL_UART_Transmit(&huart2, ...). Це забезпечило правильне перенаправлення потоку stdout у периферійний модуль USART2 мікроконтролера.

4) Фізичне блокування отворів макетної плати модулем MPU-6050

Суть проблеми: Модуль MPU-6050 має широку друковану плату; при первинному встановленні текстоліт нависав над сусідніми отворами макетної плати, через що було фізично неможливо вставити з'єднувальні дроти в ті самі горизонтальні контактні рейки.

Вирішення: Модуль було розвернуто та перенесено ніжками у крайній ряд секції (e), звільнивши доступ до ліній контактів a–d, що дозволило без перешкод підключити сигнальні дроти та живлення.

5) Складність просторового орієнтування на роз'ємі Morpho (CN10) для ліній SPI2

Суть проблеми: Роз'єм Morpho містить 38 контактів у два щільні ряди без окремих підписів кожного піна на текстоліті, через що існував ризик подати сигнали SPI (SCK, MISO, MOSI, CS) або живлення на невідповідні ніжки GPIO.

Вирішення: Було чітко локалізовано саме внутрішній ряд контактів (ближчий до чіпа STM32) і виконано прямий підрахунок пінів зверху вниз: контакт №8 для PB12 (CS), а також неперервний блок контактів №13, №14 та №15 для ліній PB15 (MOSI), PB14 (MISO) та PB13 (SCK).

6) Пошук правильної документації на компоненти

Суть проблеми: Для OV7670 було знайдено декілька версій документації, частина з яких була неповною або мала статус Preliminary.

Вирішення: Для роботи обрано повнішу версію datasheet OV7670 v1.4 та перевірено необхідні характеристики за документацією виробника/архівною копією.

7) Відмінності між мікросхемою та готовими модулями

Суть проблеми: Характеристики готових плат MPU-6050 та OV7670 не повністю збігаються з параметрами самих мікросхем, зокрема щодо живлення.

Вирішення: Окремо враховано параметри самої мікросхеми та конструкцію конкретного breakout-модуля, зокрема наявність стабілізаторів живлення на платі OV7670.

8) Формалізація ідеї проєкту у вигляді PRD

Суть проблеми: На початку роботи концепція мультирежимного реєстратора була сформована на рівні загальної ідеї, але для подальшої розробки необхідно було перетворити її на конкретний набір функцій, компонентів та критеріїв перевірки.

Було складно одразу визначити межі системи, розподілити функції між режимами Cargo та Site і пов’язати кожну функцію з конкретним апаратним компонентом.

Вирішення: Product & Documentation Lead структурував концепцію у форматі PRD: сформував функціональні вимоги FR-01–FR-17, визначив призначення компонентів та їхні альтернативи, а також підготував план тестування T-01–T-09. Це дозволило пов’язати заплановані функції системи з конкретними сенсорами, інтерфейсами та способами перевірки.

9) Побудова цілісного звіту на основі результатів командної роботи

Суть проблеми: Фінальний звіт необхідно було сформувати з нуля та об’єднати результати кількох напрямів роботи команди — налаштування build-системи, CI/CD, конфігурації STM32, апаратного підключення та дослідження компонентів. При цьому необхідно було зберегти єдину хронологічну послідовність і пов’язати опис виконаних робіт з конкретними артефактами репозиторію.

Вирішення: Product & Documentation Lead зібрав матеріали від усіх учасників, визначив послідовність етапів виконання, уніфікував технічні формулювання та структурував звіт за єдиною логікою. До відповідних етапів було додано рисунки, посилання на PR та артефакти репозиторію, після чого підготовлено фінальну версію документа у форматі PDF.

8. Підготовка фінальної документації

На завершальному етапі Product & Documentation Lead актуалізував раніше підготовлений docs/PRD.md відповідно до результатів роботи команди та узгодженої структури майбутньої системи. Основна увага була приділена узгодженню функціональних вимог FR-01–FR-17, переліку компонентів та плану тестування T-01–T-09 з фактично визначеною архітектурою проєкту. Початкову версію PRD було внесено до репозиторію та оформлено через PR #2.

.

Після завершення технічної частини роботи було зібрано результати від усіх учасників команди: конфігурацію STM32, інформацію про апаратне підключення, результати налаштування build-системи та CI/CD, а також матеріали про виявлені проблеми й способи їх вирішення. На їх основі Product & Documentation Lead сформував основну частину звіту, додав до неї рисунки, посилання на артефакти та pull request'и, систематизував опис етапів роботи та підготував фінальну структуру документа.

Завершальним результатом стала фінальна версія звіту docs/reports/report_1.pdf, яка об’єднала результати роботи всіх учасників команди. Підготовлену документацію було оформлено та внесено до репозиторію через PR #5.

9. Хронологія виконання роботи

1. **Формування концепції та планування — Product & Documentation Lead.** На початковому етапі було сформовано концепцію проєкту Incident Recorder: Cargo + Site, визначено основні сценарії використання, необхідні компоненти та загальну структуру системи. Паралельно було розподілено ролі між учасниками команди.
2. **Підготовка ізольованого середовища збірки — DevOps & Build System.** Було створено Docker-оточення на базі Ubuntu 22.04 з ARM GNU Toolchain, налаштовано кореневий Makefile та .gitignore. Після цього було перевірено локальну збірку прошивки через make build.
3. **Налаштування CI/CD — CI/CD Engineer.** Після успішної локальної збірки було створено workflow GitHub Actions для автоматичної перевірки змін. CI-конвеєр налаштовано на виконання збірки у Docker та перевірку отриманих артефактів.
4. **Дослідження апаратних компонентів — Hardware Research Engineer.** Було опрацьовано документацію на MPU-6050, W25Q64JV та OV7670, визначено

їхні інтерфейси, основні характеристики, адресацію та вимоги до підключення. Результати дослідження систематизовано у docs/hardware-components.md.

5. **Конфігурація STM32CubeMX — Team Lead / Firmware & Bring-Up.** Створено проєкт для NUCLEO-F411RE, налаштовано I2C1, SPI2, USART2, GPIO для LD2 та дерево тактування із SYSCLK = 100 МГц. Після генерації коду отримано базовий HAL-скелет прошивки.
6. **Перший запуск прошивки та перевірка UART — Team Lead / Firmware & Bring-Up.** Було реалізовано _write() для перенаправлення printf() через USART2, виконано компіляцію та прошивку NUCLEO-F411RE за допомогою st-flash. Через picocom отримано стабільний вивід діагностичних повідомлень і значень HAL_GetTick().
7. **Монтаж апаратного стенду — Team Lead / Firmware & Bring-Up.** Після успішного bring-up плати було зібрано фізичний стенд та підключено MPU-6050 через I2C1 і W25Qxx через SPI2. Під час монтажу перевірялась відповідність фізичного підключення налаштованим у CubeMX пінам та вимогам компонентів.
8. **Підготовка фінальної документації — Product & Documentation Lead.** На завершальному етапі було зібрано результати всіх учасників, систематизовано опис виконаних робіт і проблем, додано рисунки та посилання на артефакти репозиторію. Після цього було підготовлено та оформлено фінальну версію звіту docs/reports/report_1.pdf.

Висновки:

У результаті виконання лабораторної роботи ми пройшли повний базовий цикл підготовки платформи STM32 до подальшої розробки. Ми налаштували відтворюване Docker-середовище та build-конвеєр, автоматизували перевірку збірки через GitHub Actions, створили конфігурацію NUCLEO-F411RE у STM32CubeMX і розібралися з формуванням дерева тактування та налаштуванням периферії. Окремо на практиці перевірили роботу USART2, навчилися перенаправляти printf() через _write(), виконувати прошивку плати та контролювати

результат через UART. Також отримали практичний досвід підключення периферійних пристроїв через I2C та SPI і врахування особливостей їхнього фізичного підключення.

Наступним етапом для нас є перехід до лабораторної роботи № 2, де підготовлена платформа використовуватиметься вже для повноцінної роботи із сенсорами та периферійними пристроями майбутнього Incident Recorder. На основі отриманого досвіду ми приділятимемо більше уваги перевірці апаратного підключення ще до початку програмної діагностики, а також будемо одразу фіксувати технічні рішення та результати перевірок у репозиторії. Це має спростити подальше налагодження системи, інтеграцію компонентів і командну розробку.

Список використаних джерел:

1. GitHub Actions — офіційна документація
2. STM32 Nucleo-64 boards (MB1136) — User Manual
3. MPU-6000 та MPU-6050 — Register Map and Descriptions
4. MPU6050 — Datasheet
5. W25Q64JV — документація виробника Winbond
6. OV7670 — Datasheet
7. STM32CubeMX — офіційний посібник користувача UM1718
8. STM32F411xC/E — Reference Manual RM0383
9. STM32F411 — Datasheet DS10314
10. Arm GNU Toolchain — компілятор та інструменти для embedded-розробки
11. Dockerfile Reference — офіційна документація Docker
12. stlink — набір інструментів для програмування STM32 через ST-LINK
13. st-flash — документація утиліти прошивки STM32
14. picocom — документація UART-термінала